

OptSem-C: Public Optimizer Contracts for Query Engine Portability

Haoyi Zhang

Xi'an Jiaotong-Liverpool University
Suzhou, China
hyeliozhang@gmail.com

Huaijin Ran

Nanyang Technological University
Singapore, Singapore
huaijin003@e.ntu.edu.sg

ABSTRACT

Optimizer-behavior comparisons can fail before a performance number is measured: the comparison denominator is often a coarse vocabulary rather than an exact public contract. Engine-selection, migration, and performance-debugging studies often compare labels such as pushdown, join, or cost-based. Such labels can collapse local rewrites, source delegation, service-side stages, and runtime decisions into the same apparent capability, causing a study to compare different query-processing phenomena.

This paper presents OptSem-C, an executable benchmark and projection calculus for public optimizer behavior contracts. A contract rule is a guarded statement over query features with explicit public modality, operator, action variant, optimizer layer, execution placement, decision time, observability, version, and public evidence. The semantics separates behavioral worlds from evidential support, defines a conflict-aware state join, and treats lossy comparison as a projection kernel over exact contract signatures. OptSem-C lets a study publish the vocabulary it preserves, the equivalences it assumes, and the repair fields needed to make those assumptions safe.

On a source-linked corpus spanning seven analytical SQL engines, OptSem-C shows that the failure is common and repairable. Keyword and operator-only vocabularies declare thousands of engine-probe equivalences; 254 and 238 of those equivalences, respectively, are contradicted by exact public contracts. The witnesses are not a formatting effect: they survive source removal, probe-family holdouts, engine-family stress, and visible-plan validation on executable SQL probes. The missing distinctions are small and interpretable. Placement repairs keyword collisions, optimizer layer repairs operator-only collisions, and the layer+placement basis separates all 498 headline witnesses across keyword, operator-only, and yes/no projections. OptSem-C turns optimizer-contract comparison from an informal feature checklist into a finite, auditable query-processing relation.

PVLDB Reference Format:

Haoyi Zhang and Huaijin Ran. OptSem-C: Public Optimizer Contracts for Query Engine Portability. PVLDB, 20(1): XXX-XXX, 2027.
doi:XX.XX/XXX.XX

PVLDB Availability Statement:

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.
Proceedings of the VLDB Endowment, Vol. 20, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

The source code, data, and/or other artifacts have been made available at <https://doi.org/10.5281/zenodo.21092812>.

1 INTRODUCTION

SQL deliberately hides physical execution choices. That abstraction is useful for applications, but it is dangerous for optimizer comparison. The same query can be accepted by several analytical engines while exposing different public behavior: a local predicate rewrite in one engine, connector-side execution in another, a service-side stage graph in a third, and only a logical explanation in a fourth. These differences determine what a benchmark author, system integrator, or database researcher may conclude about join ordering, pushdown, materialization, adaptive execution, and plan evidence before any deployment-specific performance experiment begins.

The practical failure mode is not cosmetic. A migration checklist may equate local filter rewrite with remote-source delegation; an engine-selection study may compare compile-time join enumeration with runtime repartitioning; a debugging report may treat an estimated plan node and a service profile stage as the same evidence. OptSem-C tells such studies which public optimizer distinction they preserved and which repair fields make the comparison auditable.

Database research already has precise abstractions for the objects it chooses to study. Relational semantics starts from Codd's model [17]; Selinger-style optimizers separate access-path search from costing [61]; Volcano exposes rule-driven and parallel query processing [33]; the Volcano optimizer-generator account makes extensible search explicit [35]; and Cascades turns that line into a task-driven framework [34]. Chaudhuri surveys optimizer architecture [10]; Ioannidis surveys cost and cardinality assumptions [45]; and Steinbrunn et al. study join-order search heuristics [64]. Public optimizer behavior is usually compared with much coarser vocabularies. A cell labeled "pushdown" may mean predicate movement inside the engine, partition pruning, aggregation delegation, join delegation, limit delegation, or runtime filtering. A cell labeled "join" may refer to a logical operator, an estimated physical operator, a distributed exchange, or a runtime profile entry. A claim that two engines are "cost based" may hide whether statistics are local, connector-provided, unavailable, or revised during execution. These shortcuts are not harmless summarization: they can create projection-induced contract collisions, where a coarse vocabulary declares equality among public contracts whose exact signatures differ.

OptSem-C treats this failure as a query-processing systems problem with formal foundations. A *public optimizer contract* is the optimizer behavior an engine exposes through public plan interfaces, tuning surfaces, and system descriptions. Documentation can support a contract that is not visible in a particular textual plan, and

a plan tag can expose behavior without proving the private cost model. The object is therefore not a hidden implementation trace and not a latency measurement. OptSem-C represents each public statement as a guarded contract rule over query features. The rule records the action, the guard under which it applies, where the action is placed, which optimizer layer owns it, when the decision is made, how the behavior is observable, and whether the public contract requires, permits, forbids, or does not specify the action.

Research questions. We ask three finite, auditable questions. RQ1 asks whether public optimizer claims can be encoded as source-linked contracts and executable probes without becoming a latency benchmark. RQ2 asks how often comparison vocabularies that appear in public feature claims collapse distinct source-supported contracts. RQ3 asks which semantic fields eliminate those collisions, and whether the repair survives held-out sources, query-feature families, and engine-pair families.

The central result is a projection-kernel account of these contract collisions. Comparing optimizer contracts through a keyword or operator vocabulary applies a predeclared projection to exact signatures. If exact signatures disagree but fall into the same projected class, the projection has declared an equivalence that the public record refutes. Contract collisions are therefore semantic comparison errors whether or not a particular witness produces a latency regression; comparison validity is the prerequisite for interpreting later performance or cost claims. OptSem-C fixes the contract schema before the projection audit, computes exact and coarse classes from the same source-linked records, and reports finite repairs as certificates. Keyword projection declares 2,284 engine-probe equivalences: 2,030 have matching exact signatures and 254 are refuted. Operator-only projection declares 2,268 equivalences, including 238 collisions. Restoring placement repairs keyword collisions; restoring optimizer layer repairs operator-only collisions; restoring layer and placement repairs all 498 headline witnesses. A field-only stress test over all 256 retained-field vocabularies leaves only 44 unsafe regions, all covered by concrete counter-witnesses; after excluding action-variant identifiers, no singleton semantic field is safe.

We also introduce OptSemBench-C, a covering benchmark for optimizer-contract behavior. Its denominator is not raw query count. It is coverage of optimizer-relevant feature interactions: source boundary, connector capability, join shape, predicate class, aggregation, statistics need, reuse, distribution, adaptivity, and control surface. The generated probes are executable SQL representatives. The probe space is then cross-checked against DPccp-style join enumeration [56], JOB optimizer evidence [50], later JOB diagnostics [51], exact-cardinality testing [11], progressive optimization [54], mid-query reoptimization [48], adaptive query processing [23], and eddies [3]. We use this audit as a published-motif vocabulary stress test: it asks whether comparison words used around published workloads have concrete semantic requirements in our probe space, not whether the original studies made the same claim or measured the same performance outcome.

Contributions. (1) We define public optimizer behavior contracts as a query-processing object distinct from SQL result equivalence and runtime performance. (2) We give a guarded contract

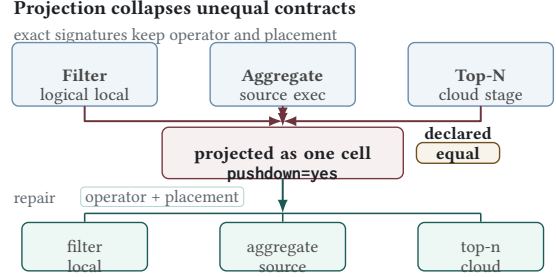


Figure 1: Projection loss in optimizer comparison. A coarse pushdown=yes label collapses unequal exact contracts; retaining operator and placement separates them.

semantics, conflict-aware state join, projection calculus, and repair-certificate workflow for publishing the assumptions of an optimizer-comparison vocabulary. (3) We introduce OptSemBench-C, a covering query-probe benchmark linked to published workload motifs, with a source-linked corpus across seven analytical SQL engines, executable SQL, and visible-plan checks. (4) We show that coarse optimizer comparisons induce contract-level collisions across a broad projection portfolio, and that the missing fields are precise query-processing semantics rather than lexical synonyms.

2 THE FAILURE MODE

Figure 1 gives the smallest example. A feature matrix maps several public optimizer contracts into the same label. After the mapping, behaviors with different placement, action kind, and evidence layer become indistinguishable, so a study that treats the label as equality can compare non-equivalent public optimizer behavior.

Feature-name collision. A cell such as pushdown=yes may denote predicate movement inside an engine, partition pruning, connector-side predicate delegation, aggregation delegation, join delegation, limit pushdown, Top-N delegation, or runtime filtering. These actions affect different operators and have different guards, placements, decision times, and observable plan effects. Collapsing them loses distinctions that matter for adaptive integration in Tukwila [46], dynamic query plans [18], outerjoin rewrites [32], compiled execution [57], Spark SQL planning [2], Calcite’s adapter-centered optimizer framework [4], Orca’s modular optimizer architecture [63], and Dremel-style serving trees [55].

Operator disappearance. A missing native plan node is not a reliable semantics. It may indicate source delegation, logical-plan observability, rewrite, stage-level aggregation, or runtime-only visibility. OptSem-C represents source-delegated execution as a positive contract action rather than as absence. This matters for modern analytical systems whose public surfaces combine Snowflake-style cloud-service stages [21], Redshift’s managed warehouse architecture [39], embedded DuckDB execution [59], vectorized MonetDB/X100 execution [6], distributed SQL services such as F1 [62], and dataflow-style execution such as Spark RDDs [69].

Architecture ranking. A local engine should not be ranked below a distributed engine merely because broadcast, shuffle, or remote-source delegation is not applicable. Conversely, a federated

or cloud engine should not be credited with local optimizer behavior when the public contract exposes delegation or service-side stages. OptSem-C separates non-applicability, documented absence, documented optional behavior, and unspecified evidence.

Observable contract versus hidden implementation. The object of study is not what an engine could do internally. It is what the public contract supports as a comparison claim. This distinction is analogous to the separation between SQL equivalence and optimizer-contract equivalence. Cosette proves SQL equivalences by reducing query semantics to logic [15]; semiring-based decision procedures give another equivalence foundation [14]; and QED expands practical SQL equivalence checking [66]. Chandra and Merlin characterize conjunctive-query containment [9]; Aho et al. characterize equivalence among relational expressions [1]. OptSem-C asks a different question: whether the public optimizer behavior being compared exposes the same evidence layers and portability obligations.

These failures imply a minimum payload for comparing public optimizer behavior: guard, canonical action, modality, placement, decision time, observability, version, and provenance. The rest of the paper formalizes this payload and measures what happens when a comparison vocabulary projects it away.

3 CONTRACT SEMANTICS

A query probe is a SQL skeleton paired with a finite feature vector and validity constraints. Features describe optimizer-relevant structure: join shape and type, predicate class, aggregation, order/limit behavior, source boundary, connector capability, statistics need, reuse structure, distribution trigger, adaptivity trigger, and optimizer control surface. They are not workload-frequency estimates; they are coordinates on the public optimizer decision surface.

DEFINITION 1 (PUBLIC CONTRACT RULE). *Let $\mathcal{A}$ be the finite domain of canonical optimizer actions. An action records operator class, action kind, optimizer layer, execution placement, decision time, observability, and variant. The state set $\mathbb{S}$ contains MUST, MAY, MUST_NOT, and UNSPEC. A public contract rule is $r = (\gamma, a, s, v, \epsilon)$, where γ is a guard over query features, $a \in \mathcal{A}$, $s \in \mathbb{S}$, v is the version or time window, and ϵ is evidence provenance. For engine e , C_e is the finite rule set, and $C_e(q) = \{r \in C_e \mid \gamma_r(\phi(q)) = \text{true}\}$ is the rule set applicable to probe q .*

The state domain has two components that feature matrices usually conflate. Behaviorally, MUST requires an action, MUST_NOT forbids it, and both MAY and UNSPEC leave it possible. Evidentially, MAY is public support for optional behavior, while UNSPEC is lack of public support. Equivalently, a state is interpreted as (m, u, e) : required behavior, allowed behavior, and evidence support. This is why documented optional pushdown and undocumented implementation possibility are different public contracts.

For each engine-probe pair, OptSem-C constructs a contract map $M_{e,q} : \mathcal{A} \rightarrow \mathbb{S}$, defaulting every action to UNSPEC. Applicable rules update the map with a conflict-aware state join $\sqcup$. The join strengthens unspecified actions when public evidence is found, preserves compatible stronger evidence, and reports contradictions instead of averaging them. The evidence component follows database-provenance practice: semiring provenance attaches support to derived facts [37], why/where provenance distinguishes

explanation locations [7], provenance surveys separate data from explanation [12], and annotated containment keeps evidence annotations visible in reasoning [36]. That separation motivates our refusal to turn missing public evidence into negative evidence.

Table 1: Conflict-aware state join for contract support states (U=UNSPEC, N=MUST_NOT, C=CONFLICT).

$\sqcup$	U	MAY	MUST	N
U	U	MAY	MUST	N
MAY	MAY	MAY	MUST	C
MUST	MUST	MUST	MUST	C
N	N	C	C	N

DEFINITION 2 (STATE-JOIN ALGEBRA). *The binary operator $\sqcup$ is the complete 4×4 partial join shown in Table 1. It is total over non-conflicting pairs and returns CONFLICT for pairs that simultaneously require and forbid, or support and forbid, the same canonical action. Let $x \leq y$ denote that $x \sqcup y = y$ for non-conflicting states. Then UNSPEC is the least informative state, while MUST and MUST_NOT are incomparable incompatible commitments.*

LEMMA 1 (MERGE INVARIANTS). *On non-conflicting states, $\sqcup$ is commutative, associative, idempotent, and monotone with respect to $\leq$. Therefore a contract map obtained by joining applicable rules is independent of rule order.*

PROOF. The properties follow by inspection of the finite table. Idempotence holds because every diagonal entry is the input state. Commutativity holds because the table is symmetric. Associativity and monotonicity are checked over the finite non-conflicting triples; the only excluded triples are exactly those containing incompatible positive and negative commitments. Lifting the pointwise operator to maps preserves the properties for every action independently. $\square$

Let $W \subseteq \mathcal{A}$ be an abstract optimizer-behavior world. A map M denotes the world set $\llbracket M \rrbracket = \{W \mid M(a) = \text{MUST} \Rightarrow a \in W, M(a) = \text{MUST_NOT} \Rightarrow a \notin W\}$. The evidential support set is $\text{Supp}(M) = \{a \in \mathcal{A} \mid M(a) \in \{\text{MUST}, \text{MAY}, \text{MUST_NOT}\}\}$, equivalently the actions where $M(a) \neq \text{UNSPEC}$. Since maps default to UNSPEC, every supported non-default entry arises from an admitted public rule. Behavioral containment and evidential support are intentionally separate: two maps can allow the same worlds while differing on whether an action is publicly documented.

DEFINITION 3 (EXACT SIGNATURES, KERNELS, AND PROJECTIONS). *The exact evidence-supported signature of M is $S(M) = \{(a, M(a)) \mid a \in \text{Supp}(M)\}$. Exact equivalence is equality of signatures over full evidence atoms. A projection is a declared map $\pi : \mathcal{A} \times \mathbb{S} \rightarrow \mathcal{B}$ from full evidence atoms to an arbitrary comparison vocabulary $\mathcal{B}$, such as keyword, yes/no, or operator-only labels. The projected signature is the finite image $S_\pi(M) = \{\pi(a, M(a)) \mid a \in \text{Supp}(M)\}$. The projection kernel is the equivalence relation $\mathcal{K}_\pi$ where $M_i \mathcal{K}_\pi M_j$ iff $S_\pi(M_i) = S_\pi(M_j)$. A projection-induced contract collision is a pair in $\mathcal{K}_\pi$ whose exact signatures differ.*

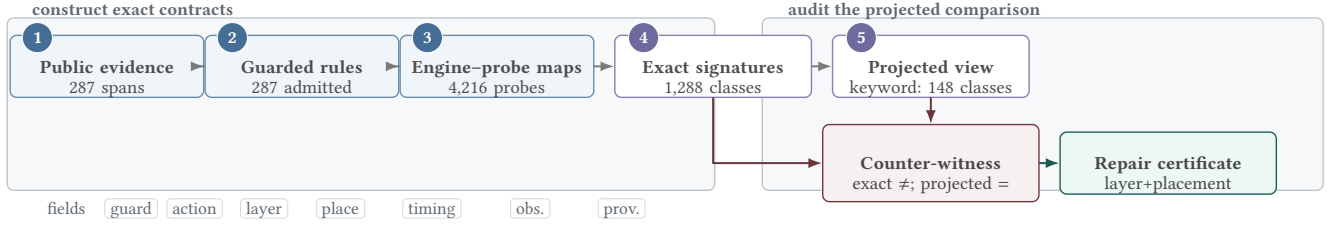


Figure 2: Contract-comparison pipeline. OptSem-C turns public evidence into exact signatures, exposes projection counter-witnesses, and reports finite repair certificates.

For quantitative comparison, exact compatibility is the Jaccard overlap $|S(M_i) \cap S(M_j)|/|S(M_i) \cup S(M_j)|$, and projected compatibility applies the same formula to S_π . The denominator is evidence-supported: undocumented actions do not count as evidence of either equality or inequality.

THEOREM 1 (PUBLIC-CONTRACT SEPARATION). *If $S(M_i) \neq S(M_j)$, then the maps differ in public-contract denotation: either $\llbracket M_i \rrbracket \neq \llbracket M_j \rrbracket$, or their evidence-supported action states differ for at least one action in $\text{Supp}(M_i) \cup \text{Supp}(M_j)$.*

PROOF. If exact signatures differ, some supported action has a state in one map that is absent or different in the other. If the difference changes a MUST or MUST_NOT requirement, the world-set constraints differ. Otherwise the behavioral world set may remain equal, but the public evidence state differs, for example MAY versus UNSPEC. In both cases the denotation differs because an OptSem-C contract is the pair of behavioral worlds and evidence-supported states. $\square$

THEOREM 2 (FINITE EVALUATION). *For finite feature and action domains, constructing $M_{e,q}$ from finite C_e and q is decidable.*

PROOF. Every guard is a finite Boolean formula over a finite feature vector. The applicable rule set is finite, and every applicable rule performs one lookup in the finite state-join table. Construction therefore terminates. $\square$

THEOREM 3 (PROJECTION-KERNEL COLLISION). *For any projection π , a projection-induced contract collision is exactly the nontrivial part of the kernel $\mathcal{K}_\pi$: M_i and M_j collide iff $S(M_i) \neq S(M_j)$ and $M_i \mathcal{K}_\pi M_j$. If π is non-injective on evidence atoms, there exist contracts with such a collision witness.*

PROOF. The first statement unfolds the definitions: $\mathcal{K}_\pi$ is equality after projection, while exact non-equivalence is $S(M_i) \neq S(M_j)$. For the non-injective case, take distinct evidence atoms $(a_1, s_1) \neq (a_2, s_2)$ with $\pi(a_1, s_1) = \pi(a_2, s_2)$. Construct M_1 so that $S(M_1) = \{(a_1, s_1)\}$ and M_2 so that $S(M_2) = \{(a_2, s_2)\}$, with all other actions UNSPEC. These minimal contracts are well formed because action states are assigned independently. Then $S(M_1) \neq S(M_2)$, but $S_\pi(M_1) = S_\pi(M_2)$ is the same singleton, so (M_1, M_2) is a collision witness. $\square$

DEFINITION 4 (PROJECTION INFORMATION PROFILE). *For a finite set of observed maps Y , let $H(S)$ be the empirical Shannon entropy of exact signatures and $H(S_\pi)$ the entropy of projected signatures. The*

entropy-retention ratio $\rho_\pi = H(S_\pi)/H(S)$ measures distinguishability retained by the comparison vocabulary. The projected collision count is the number of projected-signature classes containing more than one exact signature class.

LEMMA 2 (KERNEL INFLATION). *If a projection declares E_π equivalent pairs and exact comparison declares E over the same pair set, then E_π/E is the pairwise inflation factor induced by the kernel. Every collision witness contributes to this inflation, and a zero-collision strengthened projection must have $E_\pi = E$.*

PROOF. Exact equivalence implies projected equivalence only when the projection is a function of the exact atom payload, so $E_\pi \geq E$. The additional pairs counted by E_π are exactly the unequal exact signatures that collapse under π , which are collision witnesses. If there are none, no additional pairs are counted and the two denominators agree. $\square$

THEOREM 4 (EVIDENCE REFINEMENT). *Adding a non-conflicting evidence-backed rule cannot silently weaken a public contract: it behaviorally refines possible worlds, evidentially refines support, or both.*

PROOF. A MUST or MUST_NOT rule removes worlds from $\llbracket M \rrbracket$. A MAY rule may leave worlds unchanged, but changes an action from UNSPEC to evidence-supported. Because the rule is non-conflicting, the state join does not erase existing support or replace a compatible stronger state with a weaker one. $\square$

The remaining formal statements are a projection calculus for finite comparison audits. They are not meant to make information loss surprising; they make the certificate obligations explicit. Each reported repair records the projected equality that failed, the exact signatures that disagree, and the smaller field sets that still have counter-witnesses. Section 5 then measures which query-processing fields are empirically sufficient. Let F be the semantic-field universe and let π_R be the repaired projection that appends the field values in $R \subseteq F$ to $\pi(a, s)$ for each evidence atom (a, s) . For a finite grounded comparison set X , define $X_{\pi_R} = \{(M_i, M_j) \in X \mid S(M_i) \neq S(M_j) \wedge M_i \mathcal{K}_{\pi_R} M_j\}$. A field set R is safe, or universal, for π on X when $X_{\pi_R} = \emptyset$; otherwise it is unsafe and has a grounded counter-witness.

THEOREM 5 (PROJECTION-LATTICE FRONTIER). *If $R \subseteq R' \subseteq F$, then $X_{\pi_{R'}} \subseteq X_{\pi_R}$. Consequently the safe field sets are upward closed, the unsafe field sets are downward closed, and the minimum universal repairs form an antichain frontier in the finite field lattice.*

PROOF. Projection $\pi_{R'}$ emits every component emitted by π_R plus additional field values. Adding fields can split projected equivalence classes, but cannot merge classes already separated. Therefore any unequal pair still collapsed after adding R' was also collapsed after adding R . Upward closure of safe sets and downward closure of unsafe sets follow immediately, and minimal safe sets are incomparable by set inclusion. $\square$

For a witness $w = (M_i, M_j)$, define its separator family $\mathcal{D}_w = \{R \subseteq F \mid M_i \not\models_{\pi_R} M_j\}$. By Theorem 5, each $\mathcal{D}_w$ is upward closed in the field lattice, and universal repair is the finite intersection problem $R \in \bigcap_{w \in X_\pi} \mathcal{D}_w$.

THEOREM 6 (REPAIR CERTIFICATE). *Fix a projection π and finite comparison set X . A field set R eliminates all observed collisions iff R belongs to every witness separator family $\mathcal{D}_w$ for $w \in X_\pi$.*

PROOF. If $R \in \mathcal{D}_w$ for every witness, every pair collapsed by π becomes separated under π_R , so $X_{\pi_R} = \emptyset$. Conversely, if $X_{\pi_R} = \emptyset$, every original witness is separated under π_R , which is exactly membership in each separator family. $\square$

THEOREM 7 (RESOLUTION-LATTICE CERTIFICATE). *For a finite comparison set X and finite field universe F , exhaustive enumeration of all retained-field projections over $2^{|F|}$ subsets is a complete certificate of which public optimizer fields are sufficient for safe comparison on X . If a subset R is unsafe, a single pair $(M_i, M_j) \in X$ with $S(M_i) \neq S(M_j)$ and $S_R(M_i) = S_R(M_j)$ is a checkable counter-certificate. If R is safe, direct equality testing over all pairs in X is a checkable certificate of safety.*

PROOF. For each subset $R \subseteq F$, the retained-field signature S_R is a deterministic projection of the exact signature. The comparison set X is finite, so checking all pairs in X terminates. Unsafety is existential and is witnessed by one collapsed unequal pair. Safety is universal over the same finite set and is witnessed by the absence of such a pair after exhaustive enumeration. Since every subset of F is enumerated, the resulting safe and unsafe regions are complete for the chosen field universe. $\square$

THEOREM 8 (MINIMUM REPAIR AS HITTING SET). *For finite semantic fields F and collision witnesses X_π , deciding whether there is a universal repair of size at most k is NP-complete in $|F| + |X_\pi|$. For the fixed OptSem-C field universe, every minimum repair is exactly computable, and a certificate consists of the repaired witnesses plus counter-witnesses for every smaller field set.*

PROOF. Membership in NP follows by checking a candidate field set against every finite witness. For hardness, reduce HITTING SET. Given sets $H_1, \dots, H_m \subseteq F$, create one collision witness per H_i whose two contracts agree under the baseline projection and differ exactly on the fields in H_i . A field set repairs that witness iff it intersects H_i ; hence a universal repair of size k exists iff the hitting-set instance has a hitting set of size k . Since OptSem-C fixes a finite field universe, enumerating field subsets in increasing cardinality returns all minimum repairs. Exhausting all smaller subsets gives the lower-bound part of the certificate. $\square$

THEOREM 9 (SEMANTIC REPAIR BASIS). *Fix a finite comparison set X , projections Π , and a semantic field set F that excludes action*

identifiers such as fine-grained variants. If $R \subseteq F$ is universal for every $\pi \in \Pi$, then augmenting each projection with R eliminates all observed collisions without relying on action-identifier labels.

PROOF. Apply Theorem 6 to each projection independently. Because X and Π are finite, the union of their witness sets is finite; a common R that lies in every corresponding separator family leaves no collapsed unequal pair. Excluding action identifiers restricts the admissible field universe but does not change the certificate condition. $\square$

The semantics makes the comparison payload explicit. A user comparing only operator names chooses a projection that drops placement, decision time, variant, and observability. A feature checklist chooses an even coarser projection that can drop operator class itself. OptSem-C does not forbid those projections; it computes exactly which kernel collisions they induce, which grounded counter-witnesses remain, and which query-processing fields repair the comparison.

4 BENCHMARK AND GROUNDED CORPUS

OptSemBench-C is a behavior-contract benchmark: it generates probes that cover optimizer-relevant feature interactions and measures which public contracts those probes activate. It deliberately does not rank engines by latency. This choice follows the benchmark principle that the metric and denominator must match the object of study. CardEst asks whether cardinality estimates improve DBMS plan choice [40]; CardBench stresses learned cardinality estimators in relational settings [13]; Learned Cardinalities models correlated joins [49]; NeuroCard models probabilistic cardinalities [68]; DeepDB learns database distributions [41]; FactorJoin specializes join-cardinality estimation [67]; Neo learns plan choices [53]; and Bao evaluates learned hinting behavior [52]. An optimizer-contract benchmark must instead expose semantic traps in the comparison vocabulary.

Let $F = F_1 \times \dots \times F_n$ be the finite feature domain and Ψ the validity constraints. A target interaction is a valid partial assignment over a feature subset. The schema and feature domain are fixed before witness counting and derive from standard query-processing constructs and cited public optimizer surfaces, not from witnesses or motifs. The benchmark targets global pairwise coverage plus selected three-way interactions for source boundary, connector capability, operator class, statistics need, join shape, adaptivity trigger, and distribution trigger. A greedy generator selects renderable full assignments until all renderable target interactions are covered. The generator has two responsibilities. First, it separates benchmark adequacy from any particular system: adequacy is measured against the feature-interaction universe, not against runtime results. Second, it produces probes that are diagnostic rather than workload-frequency estimates. A probe is useful when it activates a contract guard or separates two contract signatures.

THEOREM 10 (COVERAGE). *If every valid target interaction has at least one renderable full assignment, the generator terminates with a probe set covering every renderable target interaction.*

PROOF. The target universe is finite. Each selected assignment covers at least one uncovered interaction. Thus the number of

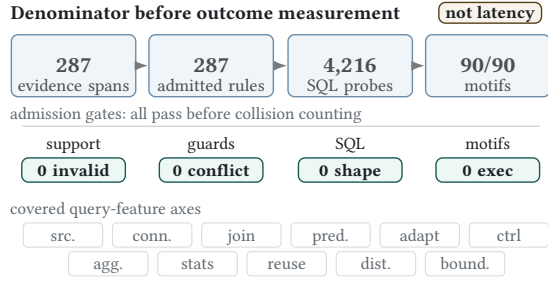


Figure 3: Corpus, benchmark, and execution denominators. The dashboard fixes evidence, SQL probes, admission gates, and motif coverage before collision rates are reported.

uncovered interactions decreases monotonically until it reaches zero. $\square$

The main empirical corpus contains only 287 source-linked grounded rules. Each rule records a public evidence span, source URL, line range, digest, and source class. Claims without grounded evidence, expressible guards, or public observability are outside the finite denominator rather than negative evidence. Thus the paper claims coverage of the declared public-contract denominator, not a census of private optimizer rules. The corpus covers plan observability, pushdown, join search, distribution, materialization, runtime adaptivity, statistics, and optimizer control surfaces. Every grounded rule is triggered by at least one generated probe, and contract merge produces no conflicts.

The source set is fixed before witness counting and includes public references for all engines in the study. PostgreSQL exposes planning and EXPLAIN surfaces [38]; Trino documents cost-based optimization and pushdown [31]; BigQuery publishes query-stage plan explanations [16]; Spark documents adaptive execution [27]; Snowflake documents logical explanation [44]; DuckDB documents CTE materialization controls [29]; and ClickHouse documents join algorithms and optimization guidance [43]. These sources are not used as performance measurements. They provide the public contracts that the framework formalizes.

The grounded corpus is intentionally conservative. A rule is admitted only when the evidence span supports an optimizer action, a guard or applicability condition, and an observability or placement interpretation. Vague performance advice is excluded from the main corpus. This policy reduces corpus size but strengthens the central claim: the evaluation is about contracts that are publicly supported, not about inferred implementation behavior.

Contract extraction and verification. Each admitted rule records the engine, version scope, source identifier, digest, action fields, guard, and observability surface. Human coding maps public evidence spans into the fixed schema; automated checks validate schema consistency, source linkage, guard reachability, trigger coverage, merge conflicts, SQL executability, and table-source consistency. Table 2 gives representative spans; the replay materials carry the complete rule-to-source manifest, source digests, claim-evidence graph, and regenerated table manifests. The field schema and projection portfolio are replay inputs fixed before collision

counting. Layer and placement are members of this predeclared field universe; exhaustive enumeration discovers the frontier, and regenerated kernel counts and repair tables validate layer+placement as a frozen semantic basis rather than a field pair chosen after inspecting one witness family. Documentation admits the public claim; EXPLAIN, profile, or stage surfaces bind exposed plan evidence when available; SQL execution validates the probe denominator rather than private optimizer truth. A counted witness requires two source-supported exact signatures, projection equality under the declared vocabulary, and separation by the reported repair field.

The denominator dashboard in Table 3 checks that this conservatism does not create unreachable contracts or syntactic probes. A finite-disjunction guard semantics evaluates each admitted rule against the generated probe denominator. All 287 grounded rules have at least one triggering probe; 252 depend on query features rather than global engine labels; and no guard uses a feature dimension or value outside the benchmark domain.

Coverage is reported over two denominators. The first is benchmark coverage: every targeted renderable valid feature interaction is covered by at least one generated probe. The second is contract triggering: every grounded rule is applicable to at least one probe. Together these denominators make the empirical scope explicit. A rule that cannot be triggered by any probe would indicate a benchmark gap; a feature interaction that cannot be rendered is excluded from the coverage denominator and reported as a modeling constraint.

The benchmark also exposes executable SQL shapes. Each generated probe has a normalized SQL skeleton and shape checks for joins, filters, grouping, ordering, limits, and reuse structure. The full 4,216-probe corpus is accompanied by a compact 48-probe motif-cover prefix. The real-engine motif execution set uses the deterministic union of this motif cover and residual feature-axis maxima; it is not a hand-picked success set, and it is not a replacement for the full evaluation.

A common failure mode in generated benchmarks is to produce feature-complete but non-executable query text. OptSemBench-C therefore validates every SQL skeleton on a deterministic relational catalog with local, remote, and cross-source namespaces. This validation is not a performance baseline and is not exhaustive contract-trigger proof. Its role is stricter and more basic: every generated probe must be parseable, plannable, executable, and shape-consistent with its feature vector. Table 3 reports the validation result: all 4,216 generated probes obtain a plan and execute, with zero SQL-shape failures. Repeating the full corpus across three deterministic catalog populations yields 12,648 successful probe-catalog executions, and production-engine checks on DuckDB and PostgreSQL add 8,432 full-corpus engine-probe executions plus 142 motif-representative executions with zero execution failures.

Contract validation is layered. Guard reachability says that each public rule has a generated probe satisfying its admitted guard; SQL execution says that the denominator is not syntactic fiction. Visible-plan sanity checks on DuckDB and PostgreSQL expose expected plan tokens at mean rates 0.705 and 0.796 over the full corpus, and 0.756 and 0.819 on motif representatives. These checks do not reverse-engineer private optimizers or prove every documented behavior fires on every cost-model instance. A counted witness still requires source-supported exact signatures, guard reachability,

Table 2: Representative source-grounded contract rules with evidence-line spans and paraphrases.

Engine	State	Action	Lines	Evidence paraphrase
PostgreSQL	MAY	Plan/observe/local	L46–L56	EXPLAIN exposes estimated startup and total cost, output rows, row width, and planner-cost units.
BigQuery	MAY	Exchange/adapt/cloud	L990–L992	The query plan may be modified during execution by adding repartition stages.
Snowflake	MAY	Plan/observe/cloud	L150–L154	EXPLAIN compiles a statement and returns a logical execution plan rather than executing it.
ClickHouse	MAY	Join/adapt/local	L313	With automatic join-algorithm selection, execution can switch from hash join to partial merge join.
DuckDB	MAY	Materialize/inline/local	L482–L487	CTE inlining is guarded by single use, absence of volatility, no grouped aggregation, and inlining hints.
Spark SQL	MAY	Plan/adapt/distributed	L129–L133	Adaptive query execution uses runtime statistics and can re-optimize a query while it is running.
Trino	MUST_NOT	Agg./pushdown/source	L168–L176	Aggregation pushdown does not support aggregate calls containing expressions.

Table 3: Benchmark denominator and validation checks across corpus, SQL, execution, and motif layers. Bold marks complete coverage or zero-failure checks.

Layer	Check	Value	Scope	Interpretation
Corpus	grounded rules	287	26 sources	finite public-contract relation
Corpus	probe-supported rules	287/287	median support 102	every admitted rule is triggerable
Corpus	non-global guards	252	3 containments	query-conditional, audited overlaps
Guard schema	invalid dimensions	0	fixed feature domain	no out-of-domain guard values
SQL corpus	generated probes	4,216	digest df107f018e37	full executable denominator
SQL corpus	motif-cover prefix	48	human inspection	compact replay slice, not cherry-picking
Execution	planned/executed probes	4,216/4,216	91 distinct plans	every generated probe plans and runs
Execution	shape/exec failures	0/0	18,448 rows	validation checks find no SQL-shape or runtime failures
Published motifs	covered requirements	90/90	12 families	every declared motif has executable probe support

Table 4: Probe budgets for triggering grounded contract rules; random columns summarize subset baselines. Bold marks complete trigger coverage.

Budget	Cover	Diagnostic	Random mean	Random 5–95
50	.672	.997	.707	[.627,.767]
100	1.000	1.000	.811	[.770,.850]
250	1.000	1.000	.890	[.864,.913]
500	1.000	1.000	.920	[.899,.941]
1000	1.000	1.000	.951	[.934,.969]
2000	1.000	1.000	.973	[.958,.986]
4216	1.000	1.000	1.000	[1.000,1.000]

projection equality, and repair separation; runtime plan satisfaction is a separate engine-specific validation layer.

The same fixed probes support two budgeted evaluation orders. The cover order is the generator’s feature-interaction order. The diagnostic order greedily maximizes newly triggered grounded rules without adding any new probe. Table 4 reports both. The cover order triggers all grounded rules within 100 probes; the diagnostic order triggers 286 of 287 rules within 50 probes and all rules within 100. Random subsets of 100 probes trigger 81% of rules on average and 85% at the 95th percentile. Thus OptSemBench-C is not merely large enough to cover the surface; it also contains compact diagnostic prefixes that expose almost all grounded contracts.

A benchmark for public optimizer contracts must connect to existing database workloads without becoming another latency benchmark. We therefore add a crosswalk from published optimizer and benchmark motifs to OptSemBench-C feature requirements. The crosswalk covers POLAR-style join-order structure [47],

TPC-H decision-support queries [20], TPC-DS decision-support queries [19], the Star Schema Benchmark [58], LDBC social-graph analytics [25], Wisconsin portability controls [5], AS3AP portability controls [65], ClickBench leaderboards [42], and LakeBench data-lake joinability [22]. Figure 4 reports the executable denominator: all 90 declared motif requirements across 12 families are covered by the generated probe space. The crosswalk is a coverage audit over optimizer-decision surfaces, not a copy of third-party benchmark SQL or a claim about third-party latency results. Here “published” means that the motif family is named in prior workload or optimizer studies outside OptSem-C; these motifs are checked after the schema is fixed and are not used to select the repair fields.

This crosswalk is also the non-circularity check for the probe denominator. The projection-kernel measurements use the OptSem-C feature vocabulary, but a counted witness must satisfy independent conditions: two public evidence-supported signatures differ, a declared comparison vocabulary collapses them, both sides have source support, and the reported repair separates the pair. The published-motif denominator is not learned from those witnesses; it asks whether third-party optimizer and workload families have concrete representatives in the same probe space. Motif requirements that cannot be represented under the current public feature domain would be reported as denominator misses rather than silently dropped. We therefore claim contract-denominator adequacy and published-motif coverage, not that generated probes reproduce third-party workload performance.

The motif-difficulty view prevents a misleading denominator. Some motifs are broad and match many probes, such as star-schema group-by shapes. Others are sparse, such as a TPC-H conjunctive-filter motif that has one matching rendered probe under the current

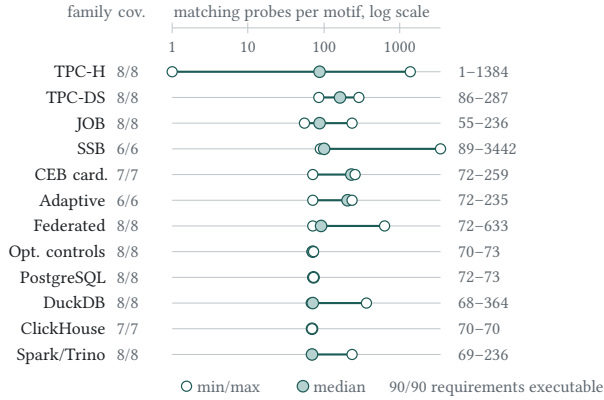


Figure 4: Published optimizer-motif denominator. The log-scale range plot shows min/median/max matching probes per family; all 90 requirements are executable.

Table 5: Matching-probe difficulty across published optimizer motifs. Underlining marks the sparsest and broadest motif coverage extremes.

Suite	Motifs	Min	Median	Max
TPC-H	8	<u>1</u>	87.0	1384
TPC-DS	8	86	164.5	287
JOB	8	55	87.0	236
SSB	6	89	101.0	<u>3442</u>
CardEst	7	72	228.0	259
Adaptive	6	72	205.5	235
Federated	8	72	91.5	633
Controls	8	70	72.0	73
PostgreSQL	8	72	73.0	73
DuckDB	8	68	72.0	364
ClickHouse	7	70	70.0	70
Spark/Trino	8	69	70.0	236

validity constraints. Reporting both coverage and matching-probe counts makes benchmark adequacy inspectable: coverage says the motif is present, while difficulty says how concentrated or redundant that coverage is. The executable motif suite adds a second denominator check: for each published motif, the system selects one satisfying probe and validates the generated SQL on the deterministic catalog, yielding 90 validated motif representatives and 383 result rows. Thus the motif denominator is not a list of names; it is a set of optimizer-decision requirements with concrete SQL representatives. Engine-plan validation is a separate experiment because the motif suite itself is a denominator check rather than a claim that published benchmark queries and generated probes induce identical plans.

5 EVALUATION

The evaluation asks four questions: how often coarse comparisons collapse unequal exact signatures, which fields repair the collisions, whether collisions reflect recognizable query-processing mechanisms rather than construction effects, and how much distinguishability each vocabulary retains. The schema, projections, and repair

Table 6: Projection-induced contract collisions and minimum repair fields for headline vocabularies. Bold marks certified repair fields; underlining marks unsafe nonzero collision evidence that must be repaired.

Projection	Eq.	False	Cond.	Repair field(s)
Keyword	2,284	<u>254</u>	<u>11.1%</u>	placement
Yes/No	2,036	6	0.3%	layer
Operator-only	2,268	<u>238</u>	10.5%	layer

search are fixed before derived tables are generated: exact signatures define the reference partition; each coarse vocabulary declares candidate equivalence classes; repairs are evaluated over all counted witnesses. The table column “false” is contract-level: it means that a declared vocabulary collapses unequal public optimizer-contract signatures. Such a collision makes the comparison denominator semantically invalid before any latency, plan-quality, or cost claim is interpreted. The baselines test comparison vocabularies, not competing optimizer implementations. Keyword, yes/no, and operator-only views are the kinds of labels exposed by engine documentation, feature matrices, workload descriptions, and plan/operator discussions; a richer comparison can always choose the strengthened surface projection and report an empty kernel when it is safe. Feature-name labels come from Trino documentation [31] and Presto documentation [30]; workload labels use TPC-H [20] and TPC-DS [19]; plan-quality controls follow JOB [50], later JOB diagnostics [51], and exact-cardinality testing [11]. Exact, ablated, one-field, and strengthened projections are controls, not additional claims about deployed systems.

Coarse comparison induces collisions. Table 6 reports the main measurement. The keyword projection declares 2,284 equivalences: 2,030 pairs agree under exact public-contract signatures and 254 pairs disagree. The operator-only projection declares 2,268 equivalences, including 238 pairs whose exact signatures disagree. These errors are conditional on the baseline declaring equivalence, which is the decision faced by a user of a feature matrix. The results are stable under binomial and probe-cluster uncertainty estimates, and the collision count remains nonzero for keyword and operator-only projections after removing any single public source. A strict-projection negative control yields zero collisions.

Counts alone understate the collision. Table 7 measures how far apart the exact contracts are after a projection has declared them equal. For the headline projections, every collision pair has exact-signature Jaccard distance 1.0: the projected comparison has equated disjoint public contract signatures rather than near duplicates. Keyword collision pairs differ by 9.09 evidence atoms on average, and operator-only collision pairs differ by 5.84 atoms. The strengthened surface projection is the control: it returns to the exact-equivalence denominator and has zero projection-induced collision.

Table 8 is the vocabulary ablation. It evaluates exact negative controls, common feature matrices, one-field adversarial vocabularies, and strengthened high-information baselines. The exact-contract baseline produces no collisions. Keyword, kind-only, and operator-only comparisons induce collisions at similar rates, so the result is

Table 7: Collision severity under lossy projections, measured by exact-signature distance and atom gaps. Bold marks the zero-collision control; underlining marks the largest unpaired unsafe residual evidence.

Projection	False	Dist.	Mean Δ	Max Δ
Keyword	254	1.00	9.09	17
Yes/No	6	1.00	3.00	3
Operator	238	1.00	5.84	14
Placement	10,702	1.00	21.29	53
State	23,593	1.00	13.06	<u>64</u>
Surface	0	0.00	0.00	0

Table 8: Executable projection baselines. Bold zeros are exact or strengthened controls; underlining marks the worst one-field projection, exposing the shown range.

Baseline	Eq.	False	Rate
Exact	2,030	0	0.0%
Keyword	2,284	254	11.1%
Op/action	2,036	6	0.3%
Operator	2,268	238	10.5%
Kind-only	2,282	252	11.0%
Layer-only	2,479	449	18.1%
Place-only	12,732	10,702	84.1%
State-only	25,623	<u>23,593</u>	<u>92.1%</u>
Surface	2,030	0	0.0%

not tied to one synonym list. One-field baselines such as placement-only, decision-time-only, and state-only are much worse, showing that a single optimizer dimension is not a safe substitute for contract semantics. By contrast, the strengthened semantic-surface baseline has zero collisions, which localizes the missing fields rather than merely declaring all abstractions bad.

Projection kernels are measurable objects. Figure 5 reports the finite information profile of representative projections. Exact signatures form 1,288 observed classes. Keyword projection compresses them into 148 projected classes, 81 of which contain multiple exact classes, retaining 64% of empirical signature entropy and inflating declared equivalence by 12.5%. Operator-only projection retains more entropy but still leaves 191 colliding projected classes and 238 pairwise collisions. Placement-only and state-only baselines demonstrate the opposite failure mode: a single field creates very large kernels. The strengthened surface projection still has projected collisions at the signature-class level, but it does not produce any pairwise collision over the engine-probe comparison set. This is the empirical counterpart of the projection-kernel theorem: the kernel is finite, populated, and repairable.

Resolution-lattice stress test. The information profile asks how much a named projection compresses exact signatures. Table 9 asks a stricter question: if a comparison is allowed to retain any subset of public optimizer fields, which retained-field vocabularies are safe? The test enumerates all 256 subsets of the eight evidence-atom fields and evaluates them over the same 88,536 engine-probe comparisons. The empty vocabulary declares 59,546 unsupported equivalences. Every singleton semantic field is unsafe: operator alone leaves 238 collisions, kind alone leaves 252, layer alone leaves 449, and placement alone leaves 10,702. Excluding action-variant identifiers, the semantic field lattice has 128 subsets, 44 unsafe

Projection kernels are measurable

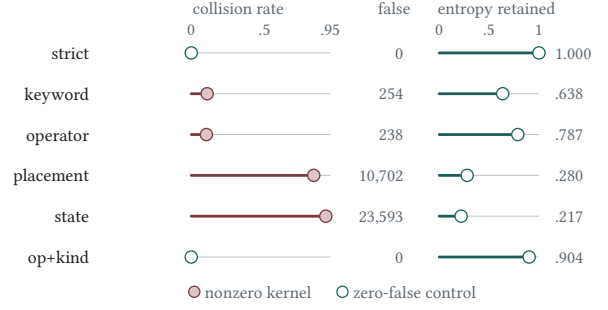


Figure 5: Projection information profile. Rose bars mark nonzero projection kernels; false-pair counts give witness denominators, while teal controls keep zero false pairs.

Table 9: Retained-field lattice for projection repair. Bold marks safe zero-collision vocabularies; underlining marks the largest unsafe collision count.

Fields	R	Classes	Ent.	Eq.	False
None	0	1	-0.000	61,576	59,546
Operator	1	387	0.787	2,268	238
Action kind	1	183	0.661	2,282	252
Layer	1	52	0.552	2,479	449
Placement	1	9	0.280	12,732	10,702
Operator+layer	2	591	0.850	2,030	0
Kind+placement	2	214	0.680	2,030	0
Surface	5	820	0.904	2,030	0
All sem.	7	947	0.934	2,030	0

regions, and minimum safe field count two; the two minimum safe no-variant vocabularies are operator+layer and kind+placement. A compact set of ten concrete engine-probe witnesses covers all unsafe retained-field regions, so the frontier is not driven by one example row. Deriving the frontier antichains gives 14 minimal safe no-variant vocabularies and eight maximal unsafe vocabularies; the monotone frontier certificate shows that every safe retained-field vocabulary contains one of the minimal safe elements and every unsafe vocabulary is contained in a maximal unsafe element.

The robustness view in Table 10 checks that the headline result is not an effect of a single denominator. The micro rate conditions on the baseline declaring equivalence; the pair-macro rate averages within engine pairs; and the leave-one-source range removes all rules supported by one public source. Keyword and operator-only collisions remain visible across these views, with leave-one-source conditional-rate ranges [0.017, 0.144] and [0.032, 0.148] respectively.

The evidence side is cross-checked without retraining on the headline witnesses. Every collision witness has at least two public source records and side-balanced support on both compared signatures. Source leave-one-out changes the public-evidence denominator, feature-family holdout learns repairs away from one query slice, and engine-family stress changes the compared engine pairs; none creates an unresolved witness after the layer+placement repair. These checks are not meant to make a perfect repair surprising by itself; they show that the same small basis survives

Table 10: Leave-one-source robustness of projection collisions across micro, pair-macro, and LOO views. Bold marks headline projections whose collisions persist under source removal.

Proj.	Decl.	False	Micro	Pair	LOO
Keyword	2284	254	0.111	0.402	[0.017,0.144]
Yes/No	2036	6	0.003	0.003	[0.000,0.052]
Operator	2268	238	0.105	0.290	[0.032,0.148]

Table 11: Dispersion of collision witnesses across probes, engine pairs, and feature values.

Projection	Wit.	Probes	Pairs	Values	Max
Keyword	254	254	3	92	1
Operator-only	238	238	2	75	1
Yes/No	6	6	1	15	1

Table 12: Finite-comparison scalability and dispersion. False is witness count; Probes/Axes count distinct probes and feature axes; Fam. is pair-family regions with witnesses out of six; Checks/s is minimum throughput in the 8× lift; Drift is mismatch against full recomputation.

Proj.	False	Probes	Axes	Fam.	Checks/s	Drift
strict	0	–	–	0/6	280,467	0
keyword	254	254	9	3/6	337,232	0
operator-only	238	238	8	2/6	365,328	0
surface	0	–	–	0/6	296,144	0

independent changes to source support, query-feature family, and engine-pair family after the schema is fixed.

The 287-rule corpus is a finite public-contract sample, not a census of optimizer internals, so the remaining checks ask whether the observed kernel is a one-source or one-shape accident. Keyword and operator-only collisions survive every leave-one-source removal; the 498 headline witnesses occupy 486 distinct probes; and the largest source affects 46 rules, 4,216 maps, and 14.3% of rule-probe rows. Table 11 further shows that witnesses touch every feature dimension and cover 92 and 75 observed feature values for keyword and operator-only projections. The witnesses are therefore dispersed projection-kernel evidence rather than duplicated rows.

Cost and scalability. Table 12 adds stress views. The finite audit executes all 88,536 engine-probe pair checks per projection, so the kernel is not sampled. The full audits complete in seconds with throughput above 50,000 comparisons per second; an 8× lift reaches 708,288 comparisons per projection with throughput above 200,000 comparisons per second and prefix-scaling $R^2 \geq 0.98$. Streaming maintenance has zero drift and uses 88,536 work units rather than the 186.7 million implied by recomputing every prefix. A source refresh is local: the largest public source affects 46 rules, 4,216 maps, and 14.3% of rule-probe map rows. The table also ties scaling to dispersion and family stratification: keyword collisions span 254 probes, nine feature axes, and three of six pair-family regions; operator-only spans 238 probes, eight axes, and two regions.

Table 13: Repair fields versus attribution leaders across headline projections.

Projection	Single-field repairs	Top attribution fields
Keyword	variant, placement	variant=,168; placement=,168
Yes/No	variant, layer, placement, state	each=,250
Operator-only	variant, layer	variant=,184; layer=,184

The probe-subsample view checks that the collision result is not an effect of using the full generated benchmark. At 10% and 50% probe subsamples, keyword and operator-only projections retain at least one collision in every trial, with median conditional rates close to the full benchmark. The weaker yes/no projection is sparse at 10%, but becomes nonzero in every trial by 50%.

The mutation-suite view is adversarial: it expands the comparison vocabulary around the headline baselines rather than only evaluating the three projections used in the abstract. Exact signatures remain a zero-false negative control, one-field projections produce much larger kernels, and strengthened projections that restore operator, action kind, and the public semantic surface collapse to the exact-equivalence denominator. The result is not that every abstraction is bad; it is that common optimizer-comparison abstractions omit query-processing fields that several independently weakened projections need and several strengthened projections restore.

Repair certificates identify the missing semantics. The projection calculus makes repair checkable, but the empirical question is which fields are sufficient. The repair column in Table 6 reports minimum-cardinality universal repairs and held-out-probe generalization. Placement repairs every keyword collision. Layer repairs every operator-only collision. The learned repairs resolve all held-out collisions across probe folds. A fixed semantic basis consisting of optimizer layer and execution placement repairs all observed collisions across keyword, yes/no, and operator-only projections, even when fine-grained action-variant labels are excluded. The certificate is checked as a finite proof obligation: each counted witness is full-signature unequal, projection-equal, separated by the reported repair, and protected by a grounded counter-witness against every smaller universal repair. The hitting-set view over all 498 headline witnesses agrees exactly with direct repaired-signature enumeration.

Table 15 adds a stronger anti-overfitting test. Each fold withholds a non-default query-feature family, learns repairs from the remaining collision witnesses, and tests them on the held-out family. A point minimum can choose an underspecified field when the training denominator is narrow; the fixed layer and placement basis resolves every held-out collision across all folds. The repair is therefore not a post-hoc fit to one query slice, but a stable contract denominator.

The repair ladder in Table 14 gives a stepwise view of the same diagnosis. Restoring operator alone nearly repairs keyword projection but leaves six collisions; placement eliminates them. For operator-only projection, layer eliminates all collisions while placement alone leaves residual errors. Figure 6 then reports the finite field-lattice frontier: safe subsets repair every collision in scope, unsafe

Table 14: Projection repair ladder showing residual false pairs after restoring fields. Bold marks certified zero-collision repairs; underlining marks unrepaired unsafe baseline rows.

Projection	Restored fields	Eq.	False
Keyword	–	2,284	<u>254</u>
Keyword	operator	2,036	6
Keyword	layer	2,072	42
Keyword	placement	2,030	0
Keyword	layer+placement	2,030	0
Yes/No	–	2,036	6
Yes/No	layer	2,030	0
Yes/No	placement	2,030	0
Operator	–	<u>2,268</u>	<u>238</u>
Operator	kind	2,036	6
Operator	placement	2,062	32
Operator	layer	2,030	0
Operator	layer+placement	2,030	0

Table 15: Held-out feature-family repair stability for point minima and the fixed basis. Bold marks zero held-out residuals; underlining marks unrepaired unsafe residual evidence under underspecified point minima.

Projection	Folds	Held-out	Point	Basis
Keyword	12	830	<u>144</u>	0
Operator	9	569	<u>12</u>	0
Yes/No	1	3	0	0

Repair frontier over retained fields					
scope	false	depth	minimum basis	safe/unsafe	
		0 1 2+			
keyword	254	U S 	placement	23/9	
yes/no	6	U S 	layer or placement	24/8	
operator-only	238	U S 	layer	22/10	
all	498	U U S	layer + placement	21/11	
		U unsafe S first safe			

Figure 6: Semantic frontier over retained fields. Rose U cells leave unsafe counter-witnesses; teal S cells mark first safe retained-field sets, with layer+placement required jointly.

subsets retain grounded counter-witnesses, and the all-projection row requires layer+placement for all 498 headline witnesses.

Table 13 shows why the repair fields are query-processing fields. Placement separates local execution, source delegation, distributed workers, and cloud-service execution. Layer and decision time separate logical rewrite, physical planning, distributed planning, and runtime adaptation. Operator and observability keep plan operators, plan evidence, physical estimates, and runtime profiles from being treated as interchangeable. The highest-attribution fields are the same fields that repair the collisions, and a fixed layer+placement semantic basis resolves every observed engine-pair hotspot without relying on fine-grained variant identifiers.

Table 16: Collision mechanisms and representative engine-pair hotspots.

Mechanism	W.	Hotspot	False
Action variant	498	Keyword BQ-CH	203
Placement	466	Keyword BQ-Trino	45
Optimizer layer	430	Keyword CH-Trino	6
Plan evidence	366	Yes/No CH-Trino	6
Decision time	362	Operator CH-Trino	123
Operator family	281	Operator Spark-Trino	115

Table 17: Workload-feature contract movements under controlled toggles. Bold marks the largest mean movement; underlining marks the broadest nonzero movement.

Engine	Feature	Mean dist.	Nonzero
DuckDB	reuse_structure	0.827	0.854
Trino	statistics_need	0.742	0.853
DuckDB	source_boundary	0.590	0.623
Trino	distribution_trigger	0.519	0.861
Spark SQL	adaptivity_trigger	0.442	0.858
BigQuery	distribution_trigger	0.373	<u>0.891</u>
Spark SQL	join_type	0.369	0.810
ClickHouse	order_limit	0.351	0.511

Collisions have mechanisms and hotspots. Table 16 groups collision witnesses into optimizer mechanisms and lists the largest engine-pair hotspots. The dominant mechanisms are action-variant, execution-placement, optimizer-layer, plan-evidence, and decision-time collisions. The hotspots are not random; they occur where public contracts expose different query-processing surfaces, such as source delegation versus local planning, runtime adaptivity versus compile-time planning, and stage-graph observability versus estimated local plans.

Optimizer contracts are workload-sensitive. Table 17 reports controlled feature-toggle movements. The largest movements involve CTE reuse, source boundary, statistics need, distribution triggers, and adaptivity triggers. Optimizer-contract comparability is therefore not an engine-level scalar; it is a relation between engine contracts and query structure.

The evaluation links theory and evidence: the nontrivial kernel is populated by public optimizer evidence rather than synthetic examples. The information profile, frontier certificate, and SQL validation show that the denominator is distinguishable, repairable, and executable on deterministic catalogs plus DuckDB and PostgreSQL.

5.1 Grounded Case Studies

Table 18 gives the concrete rule pairs behind the aggregate measurements. The rows span the major mechanism families in Table 16, not unusually easy repairs. C1 expands a remote-source aggregate probe with grouping and regional filtering. BIGQUERY-FEDERATED contributes source-delegated filtering plus a CloudSQL blocking condition; TRINO-PUSH contributes aggregate pushdown, aggregate-expression blocking, and join-pushdown contracts. Keyword projection sees “pushdown” in both maps, so $S_\pi(M_{BQ}) = S_\pi(M_{Trino})$, but exact signatures separate filter delegation with stage evidence from aggregate delegation plus aggregate-expression

Table 18: Worked public-contract cases linking coarse labels to hidden contract pairs and protected downstream decisions.

Case	Coarse label	Contract pair hidden by the label	Separating fields	Decision protected
C1	pushdown=yes	BigQuery federated filter delegation vs. Trino aggregate/expression pushdown	operator, variant, placement	do not port local rewrite expectations to delegated execution
C2	plan evidence	PostgreSQL EXPLAIN estimates vs. BigQuery stage/shuffle evidence and Snowflake logical plans	observability, placement	do not compare estimates, stages, and profiles as one evidence layer
C3	adaptive=yes	Compile-time planning vs. BigQuery/Spark/ClickHouse runtime adaptation	layer, decision time	do not benchmark runtime adaptation as compile-time search
C4	CTE support	DuckDB/PostgreSQL CTE inline, materialize, reuse, and disable guards	variant, guard state	do not treat guarded reuse as unconditional feature parity

denial. The certificate checks exact inequality, projection equality, separation after retaining placement and operator/action variant, and a smaller-field counter-witness.

C2 separates PostgreSQL compile-time estimates [38] from BigQuery stages, shuffle metrics, and profile-time evidence [16], and from Snowflake logical-plan evidence [44]. The protected decision is whether an engine-selection report may compare estimates, service-stage graphs, and profiles as one evidence layer. C3 separates compile-time choices from runtime adaptation: BigQuery repartition [16], Spark AQE [27], and ClickHouse automatic join conversion [43] differ from static join selection, so adaptation claims must retain layer and decision time. C4 contrasts DuckDB inline, materialize, and reuse guards [29] with PostgreSQL common-table-expression controls [38]; the repair changes a migration checklist from feature parity to guard-dependent obligations. These cases are representative: the same rule schema and finite certificate checker cover all 498 headline witnesses.

6 DISCUSSION AND LIMITATIONS

Optimizer interfaces as semantic interfaces. A public optimizer claim should name the compared action, guard, layer, placement, decision time, and evidence surface. These fields do not expose proprietary cost formulas; they prevent heterogeneous engines, adapters, remote sources, and service-side stages from being treated as one phenomenon. This matters in Garlic-style federation [8] and BigDAWG-style polystores [24], where one user operation crosses engine and source boundaries.

Protocol, scope, and validity. Every optimizer comparison is a projection: choose a vocabulary, extract public contracts, validate the denominator, compute the kernel, and publish retained fields or assumptions. The evidence unit is a public contract, not inferred private state; absent manuals, plans, diagnostics, connector behavior, or observable SQL behavior, a rule is outside scope unless it changes that contract. Because documentation can be incomplete or stale, OptSem-C separates source digests, version scope, and visible-plan checks; a document-plan disagreement is a contract-evidence conflict, not a private-truth correction. Motif crosswalks, source leave-one-out tests, feature holdouts, engine-family stress, and real-engine SQL checks mitigate circularity by changing the denominator, support, or execution surface.

Using repair certificates. A repair certificate changes how comparisons are stated. Instead of saying that two systems both “support pushdown” or “use adaptive optimization,” a study can state the certified frontier: local versus delegated execution, logical versus physical planning, compile-time versus runtime choice, and

plan versus contract evidence. Version scope and evidence digests make source refresh an incremental audit.

7 RELATED WORK

Optimizer architecture and extensible planning. Classical optimizers separate relational equivalence, access paths, costing, and search: System R exposes access-path selection [61], Chaudhuri surveys optimizer pipelines [10], and Ioannidis surveys cost assumptions [45]. Volcano structures rule-driven processing [33]; the optimizer-generator paper generalizes construction [35]; and Cascades casts search as tasks [34]. Calcite traits and adapters [4, 26], Orca modules [63], DataFusion plans [28], and Presto settings [30] make public-comparison fields observable.

Equivalence, testing, and provenance. Cosette [15], semiring-based SQL equivalence [14], and QED [66] decide result preservation, while NoREC exposes optimizer bugs [60]. Chandra and Merlin [9] and Aho et al. [1] characterize containment and expression equivalence. OptSem-C addresses a different question: result-equivalent plans can still expose different public optimizer evidence. Its evidence model follows provenance semirings [37], why/where provenance [7], provenance surveys [12], and annotated-query containment [36].

Optimizer benchmarks and adaptive systems. JOB makes the plan-quality denominator explicit [50], later JOB analysis exposes optimizer behavior [51], and exact-cardinality testing isolates choices [11]. Progressive optimization [54], mid-query re-optimization [48], adaptive query processing [23], and eddies [3] move decisions to runtime. Learned optimizers choose or rank plans [52, 53, 70], while Learned Cardinalities [49], NeuroCard [68], and DeepDB [41] model distributions. OptSem-C asks whether compared behavior has the same public contract denominator.

Workload and system benchmarks. TPC-H [20] and TPC-DS [19] define decision-support denominators; SSB [58], LDBC [25], LakeBench [22], and CardBench [13] target warehouses, graph analytics, data-lake joins, and learned cardinalities. Wisconsin [5], AS3AP [65], and ClickBench [42] add portability and scale views. OptSemBench-C keeps that discipline but checks optimizer-facing motifs against public contracts.

8 CONCLUSION

A coarse optimizer comparison can declare equivalence where public contracts disagree. OptSem-C formalizes that failure with finite projections over 287 rules and 4,216 probes. In this corpus, layer+placement separates all 498 headline witnesses, and every unsafe retained-field region has a concrete counter-witness.

REFERENCES

- [1] A. V. Aho, Y. Sagiv, and J. D. Ullman. 1979. Equivalences among Relational Expressions. *SIAM J. Comput.* 8, 2 (1979), 218–246. <https://doi.org/10.1137/0208017>
- [2] Michael Armbrust, Reynold S. Xin, Cheng Lian, Yin Huai, Davies Liu, Joseph K. Bradley, Xiangrui Meng, Tomer Kaftan, Michael J. Franklin, Ali Ghodsi, and Matei Zaharia. 2015. Spark SQL: Relational Data Processing in Spark. In *SIGMOD*. 1383–1394. <https://doi.org/10.1145/2723372.2742797>
- [3] R. Avnur and J. M. Hellerstein. 2000. Eddies: Continuously Adaptive Query Processing. In *SIGMOD*. 261–272. <https://doi.org/10.1145/342009.335420>
- [4] E. Begoli, J. Camacho-Rodriguez, J. Hyde, M. J. Mior, and D. Lemire. 2018. Apache Calcite: A Foundational Framework for Optimized Query Processing Over Heterogeneous Data Sources. In *SIGMOD*. 221–230. <https://doi.org/10.1145/3183713.3190662>
- [5] D. Bitton, D. J. DeWitt, and C. Turbyfill. 1983. Benchmarking Database Systems: A Systematic Approach. In *Vldb*. 8–19. <https://www.vldb.org/conf/1983/P008.PDF>
- [6] P. A. Boncz, M. Zukowski, and N. Nes. 2005. MonetDB/X100: Hyper-Pipelining Query Execution. In *CIDR*. 225–237. <https://www.cidrdb.org/cidr2005/papers/P19.pdf>
- [7] P. Buneman, S. Khanna, and W.-C. Tan. 2001. Why and Where: A Characterization of Data Provenance. In *ICDT*. 316–330. https://doi.org/10.1007/3-540-44503-X_20
- [8] Michael J. Carey, Laura M. Haas, Peter M. Schwarz, Manish Arya, William F. Cody, Ronald Fagin, Myron Flickner, Allen W. Luniewski, Wayne Niblack, Dragutin Petkovic, John Thomas, John H. Williams, and Edward L. Wimmers. 1995. Towards Heterogeneous Multimedia Information Systems: The Garlic Approach. In *RIDE-DOM*. 124–131. <https://doi.org/10.1109/RIDE.1995.378736>
- [9] A. K. Chandra and P. M. Merlin. 1977. Optimal Implementation of Conjunctive Queries in Relational Data Bases. In *STOC*. 77–90. <https://doi.org/10.1145/800105.803397>
- [10] S. Chaudhuri. 1998. An Overview of Query Optimization in Relational Systems. In *PODS*. 34–43. <https://doi.org/10.1145/275487.275492>
- [11] S. Chaudhuri, V. R. Narasayya, and R. Ramamurthy. 2009. Exact Cardinality Query Optimization for Optimizer Testing. *PVLDB* 2, 1 (2009), 994–1005. <https://doi.org/10.14778/1687627.1687739>
- [12] J. Cheney, L. Chiticariu, and W.-C. Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474. <https://doi.org/10.1561/1900000006>
- [13] Y. Chronis, Y. Wang, Y. Gan, S. Abu-El-Haija, C. Lin, C. Binnig, and F. Özcan. 2024. CardBench: A Benchmark for Learned Cardinality Estimation in Relational Databases. *arXiv*. arXiv:2408.16170 <https://arxiv.org/abs/2408.16170>
- [14] S. Chu, B. Murphy, J. Roesch, A. Cheung, and D. Suciu. 2018. Axiomatic Foundations and Algorithms for Deciding Semantic Equivalences of SQL Queries. *PVLDB* 11, 11 (2018), 1482–1495. <https://doi.org/10.14778/3236187.3236200>
- [15] S. Chu, C. Wang, K. Weitz, and A. Cheung. 2017. Cosette: An Automated Prover for SQL. In *CIDR*. 1–10. <https://www.cidrdb.org/cidr2017/papers/p51-chu-cidr17.pdf>
- [16] Google Cloud. 2026. Query Plan and Timeline. Documentation. <https://cloud.google.com/bigquery/docs/query-plan-explanation>
- [17] E. F. Codd. 1970. A Relational Model of Data for Large Shared Data Banks. *Commun. ACM* 13, 6 (1970), 377–387. <https://doi.org/10.1145/362384.362685>
- [18] R. L. Cole and G. Graefe. 1994. Optimization of Dynamic Query Evaluation Plans. In *SIGMOD*. 150–160. <https://doi.org/10.1145/191839.191872>
- [19] Transaction Processing Performance Council. 2024. TPC Benchmark DS Standard Specification. TPC. <https://www.tpc.org/tpcds/>
- [20] Transaction Processing Performance Council. 2024. TPC Benchmark H Standard Specification. TPC. <https://www.tpc.org/tpch/>
- [21] Benoit Dageville, Thierry Cruanes, Marcin Zukowski, Vadim Antonov, Artin Avanes, Jon Bock, Jonathan Claybaugh, Daniel Engovatov, Martin Hentschel, Jiansheng Huang, Allison W. Lee, Ashish Motivala, Abdul Q. Munir, Steven Pelley, Peter Povinec, Greg Rahn, Spyridon Triantafyllis, and Philipp Unterbrunner. 2016. The Snowflake Elastic Data Warehouse. In *SIGMOD*. 215–226. <https://doi.org/10.1145/2882903.2903741>
- [22] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *PVLDB* 17, 8 (2024), 1925–1938. <https://doi.org/10.14778/3659437.3659448>
- [23] A. Deshpande, Z. Ives, and V. Raman. 2007. Adaptive Query Processing. *Foundations and Trends in Databases* 1, 1 (2007), 1–140. <https://doi.org/10.1561/1900000001>
- [24] Jennie Duggan, Aaron J. Elmore, Michael Stonebraker, Magda Balazinska, Bill Howe, Jeremy Kepner, Sam Madden, David Maier, Tim Mattson, and Stan Zdonik. 2015. The BigDAWG Polystore System. *SIGMOD Record* 44, 2 (2015), 11–16. <https://doi.org/10.1145/2814710.2814713>
- [25] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC Social Network Benchmark: Interactive Workload. In *SIGMOD*. 619–630. <https://doi.org/10.1145/2723372.2742786>
- [26] Apache Software Foundation. 2026. Apache Calcite Documentation. Documentation. <https://calcite.apache.org/docs/>
- [27] Apache Software Foundation. 2026. Performance Tuning: Spark 4.1.2 Documentation. Documentation. <https://spark.apache.org/docs/4.1.2/sql-performance-tuning.html>
- [28] Apache Software Foundation. 2026. Reading Explain Plans. Documentation. <https://datafusion.apache.org/user-guide/explain-usage.html>
- [29] DuckDB Foundation. 2026. DuckDB Documentation: WITH Clause and CTE Materialization. Documentation. https://duckdb.org/docs/stable/sql/query_syntax/with.html
- [30] Presto Foundation. 2026. Cost Based Optimizations: Presto 0.298.1 Documentation. Documentation. <https://prestodb.github.io/docs/current/optimizer/cost-based-optimizations.html>
- [31] Trino Software Foundation. 2026. Cost-Based Optimizations: Trino 482 Documentation. Documentation. <https://trino.io/docs/482/optimizer/cost-based-optimizations.html>
- [32] Cesar A. Galindo-Legaria and Arnon Rosenthal. 1997. Outerjoin Simplification and Reordering for Query Optimization. *ACM Transactions on Database Systems* 22, 1 (1997), 43–74. <https://doi.org/10.1145/244810.244812>
- [33] G. Graefe. 1990. Encapsulation of Parallelism in the Volcano Query Processing System. In *SIGMOD*. 102–111. <https://doi.org/10.1145/93597.98720>
- [34] G. Graefe. 1995. The Cascades Framework for Query Optimization. *IEEE Data Engineering Bulletin* 18, 3 (1995), 19–29. <https://www.sigmod.org/publications/dblp/db/journals/debu/Graefe95a.html>
- [35] G. Graefe and W. J. McKenna. 1993. The Volcano Optimizer Generator: Extensibility and Efficient Search. In *ICDE*. 209–218. <https://doi.org/10.1109/ICDE.1993.344061>
- [36] T. J. Green. 2011. Containment of Conjunctive Queries on Annotated Relations. *Theory of Computing Systems* 49, 2 (2011), 429–459. <https://doi.org/10.1007/s00224-011-9327-6>
- [37] T. J. Green, G. Karvounarakis, and V. Tannen. 2007. Provenance Semirings. In *PODS*. 31–40. <https://doi.org/10.1145/1265530.1265535>
- [38] PostgreSQL Global Development Group. 2026. PostgreSQL Documentation: 18: 14.1. Using EXPLAIN. Documentation. <https://www.postgresql.org/docs/18/using-explain.html>
- [39] Anurag Gupta, Deepak Agarwal, Derek Tan, Jakub Kulesza, Rahul Pathak, Stefano Stefani, and Vidhya Srinivasan. 2015. Amazon Redshift and the Case for Simpler Data Warehouses. In *SIGMOD*. 1917–1923. <https://doi.org/10.1145/2723372.2742795>
- [40] Yuxing Han, Ziniu Wu, Peizhi Wu, Rong Zhu, Jingyi Yang, Liang Wei Tan, Kai Zeng, Gao Cong, Yanzhao Qin, Andreas Pfadler, Zhengping Qian, Jingren Zhou, Jiangneng Li, and Bin Cui. 2021. Cardinality Estimation in DBMS: A Comprehensive Benchmark Evaluation. *PVLDB* 15, 4 (2021), 752–765. <https://doi.org/10.14778/3503585.3503586>
- [41] B. Hilprecht, A. Schmidt, M. Kulesa, A. Molina, K. Kersting, and C. Binnig. 2020. DeepDB: Learn from Data, Not from Queries! *PVLDB* 13, 7 (2020), 992–1005. <https://doi.org/10.14778/3384345.3384349>
- [42] ClickHouse Inc. 2024. ClickBench: A Benchmark for Analytical DBMS. Online benchmark. <https://benchmark.clickhouse.com/>
- [43] ClickHouse Inc. 2026. Using JOINS in ClickHouse. Documentation. <https://clickhouse.com/docs/guides/joining-tables>
- [44] Snowflake Inc. 2026. EXPLAIN: Snowflake Documentation. Documentation. <https://docs.snowflake.com/en/sql-reference/sql/explain>
- [45] Y. E. Ioannidis. 1996. Query Optimization. *Comput. Surveys* 28, 1 (1996), 121–123. <https://doi.org/10.1145/234313.234367>
- [46] Z. G. Ives, D. Florescu, M. Friedman, A. Levy, and D. S. Weld. 1999. An Adaptive Query Execution System for Data Integration. *ACM SIGMOD Record* 28, 2 (1999), 299–310. <https://doi.org/10.1145/304181.304209>
- [47] David Justen, Daniel Ritter, Campbell Fraser, Andrew Lamb, Allison Lee, Thomas Bodner, Mhd Yamen Haddad, Steffen Zeuch, Volker Markl, and Matthias Boehm. 2024. POLAR: Adaptive and Non-invasive Join Order Selection via Plans of Least Resistance. *PVLDB* 17, 6 (2024), 1350–1363. <https://doi.org/10.14778/3648160.3648175>
- [48] N. Kabra and D. J. DeWitt. 1998. Efficient Mid-Query Re-Optimization of Sub-Optimal Query Execution Plans. In *SIGMOD*. 106–117. <https://doi.org/10.1145/276304.276315>
- [49] Andreas Kipf, Thomas Kipf, Bernhard Radke, Viktor Leis, Peter Boncz, and Alfons Kemper. 2019. Learned Cardinalities: Estimating Correlated Joins with Deep Learning. In *CIDR*. 1–11. <https://www.cidrdb.org/cidr2019/papers/p101-kipf-cidr19.pdf>
- [50] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2015. How Good Are Query Optimizers, Really? *PVLDB* 9, 3 (2015), 204–215. <https://doi.org/10.14778/2850583.2850594>

- [51] V. Leis, B. Radke, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann. 2018. Query Optimization Through the Looking Glass, and What We Found Running the Join Order Benchmark. *VLDB Journal* 27, 5 (2018), 643–668. <https://doi.org/10.1007/s00778-017-0480-7>
- [52] Ryan Marcus, Parimarjan Negi, Hongzi Mao, Nesime Tatbul, Mohammad Alizadeh, and Tim Kraska. 2021. Bao: Making Learned Query Optimization Practical. In *SIGMOD*. 1275–1288. <https://doi.org/10.1145/3448016.3452838>
- [53] R. Marcus and O. Papaemmanouil. 2019. Neo: A Learned Query Optimizer. *PVLDB* 12, 11 (2019), 1705–1718. <https://doi.org/10.14778/3342263.3342644>
- [54] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, and M. Cilimdžić. 2004. Robust Query Processing Through Progressive Optimization. In *SIGMOD*. 659–670. <https://doi.org/10.1145/1007568.1007642>
- [55] Sergey Melnik, Andrey Gubarev, Jing Jing Long, Geoffrey Romer, Shiva Shivakumar, Matt Tolton, and Theo Vassilakis. 2010. Dremel: Interactive Analysis of Web-Scale Datasets. *PVLDB* 3, 1-2 (2010), 330–339. <https://doi.org/10.14778/1920841.1920886>
- [56] G. Moerkotte and T. Neumann. 2006. Analysis of Two Existing and One New Dynamic Programming Algorithm for the Generation of Optimal Bushy Join Trees without Cross Products. In *Vldb*. 930–941. <https://dblp.org/rec/conf/vldb/MoerkotteN06.html>
- [57] T. Neumann. 2011. Efficiently Compiling Efficient Query Plans for Modern Hardware. *PVLDB* 4, 9 (2011), 539–550. <https://doi.org/10.14778/2002938.2002940>
- [58] P. O’Neil, E. O’Neil, X. Chen, and S. Revilak. 2009. The Star Schema Benchmark and Augmented Fact Table Indexing. In *TPCTC*. 237–252. https://doi.org/10.1007/978-3-642-10424-4_17
- [59] M. Raasveldt and H. Muehleisen. 2019. DuckDB: An Embeddable Analytical Database. In *SIGMOD*. 1981–1984. <https://doi.org/10.1145/3299869.3320212>
- [60] M. Rigger and Z. Su. 2020. Detecting Optimization Bugs in Database Engines via Non-Optimizing Reference Engine Construction. In *ESEC/FSE*. 1140–1152. <https://doi.org/10.1145/3368089.3409710>
- [61] P. G. Selinger, M. M. Astrahan, D. D. Chamberlin, R. A. Lorie, and T. G. Price. 1979. Access Path Selection in a Relational Database Management System. In *SIGMOD*. 23–34. <https://doi.org/10.1145/582095.582099>
- [62] Jeff Shute, Radek Vingralek, Bart Samwel, Ben Handy, Chad Whipkey, Eric Rollins, Mircea Oancea, Kyle Littlefield, David Menestrina, Stephan Ellner, John Cieslewicz, Ian Rae, Traian Stancescu, and Himani Apte. 2013. F1: A Distributed SQL Database That Scales. *PVLDB* 6, 11 (2013), 1068–1079. <https://doi.org/10.14778/2536222.2536232>
- [63] Mohamed A. Soliman, Lyublena Antova, Venkatesh Raghavan, Amr El-Helw, Zhongxian Gu, Entong Shen, George C. Caragea, Carlos Garcia-Alvarado, Foyzur Rahman, Michalis Petropoulos, Florian Waas, Sivaramakrishnan Narayanan, Konstantinos Krikellias, and Rhonda Baldwin. 2014. Orca: A Modular Query Optimizer Architecture for Big Data. In *SIGMOD*. 337–348. <https://doi.org/10.1145/2588555.2595637>
- [64] M. Steinbrunn, G. Moerkotte, and A. Kemper. 1997. Heuristic and Randomized Optimization for the Join Ordering Problem. *Vldb Journal* 6, 3 (1997), 191–208. <https://doi.org/10.1007/s007780050040>
- [65] C. Turbyfill, C. Orji, and D. Bitton. 1993. AS3AP: An ANSI SQL Standard Scaleable and Portable Benchmark for Relational Database Systems. In *The Benchmark Handbook for Database and Transaction Processing Systems*. 95–121. <https://www.odbm.org/2014/03/benchmark-handbook-1993/>
- [66] S. Wang, S. Pan, and A. Cheung. 2024. QED: A Powerful Query Equivalence Decoder for SQL. *PVLDB* 17, 11 (2024), 3602–3614. <https://doi.org/10.14778/3681954.3682024>
- [67] Z. Wu, P. Negi, M. Alizadeh, T. Kraska, and S. Madden. 2023. FactorJoin: A New Cardinality Estimation Framework for Join Queries. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–27. <https://doi.org/10.1145/3588721>
- [68] Z. Yang, E. Liang, A. Kamsetty, C. Wu, Y. Duan, X. Chen, P. Abbeel, J. M. Hellerstein, S. Krishnan, and I. Stoica. 2019. Deep Unsupervised Cardinality Estimation. *PVLDB* 13, 3 (2019), 279–292. <https://doi.org/10.14778/3368289.3368294>
- [69] Matei Zaharia, Mosharaf Chowdhury, Tathagata Das, Ankur Dave, Justin Ma, Murphy McCauley, Michael J. Franklin, Scott Shenker, and Ion Stoica. 2012. Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In *NSDI*. 15–28. <https://www.usenix.org/conference/nsdi12/technical-sessions/presentation/zaharia>
- [70] R. Zhu, W. Chen, B. Ding, X. Chen, A. Pfadler, Z. Wu, and J. Zhou. 2023. Lero: A Learning-to-Rank Query Optimizer. *PVLDB* 16, 6 (2023), 1466–1479. <https://doi.org/10.14778/3583140.3583160>